OR-Tools — `ortools.sat.python.cp_model`

CLASES PRINCIPALES

`cp.CpModel()`
Crea el modelo de restricciones

`cp.CpSolver()`
Instancia el solver

`cp.CpSolverSolutionCallback`
Clase base para capturar soluciones

DECLARACIÓN DE VARIABLES

`model.NewIntVar(lb, ub, nombre)`
Variable entera con dominio [lb, ub]

`model.NewBoolVar(nombre)`
Variable booleana 0/1

`model.NewIntVar(v, v, nombre)`
Variable fija (lb == ub)

`model.NewIntVarFromDomain(d, n)` +
Variable desde un dominio explícito

RESTRICCIONES — USADAS EN LOS EJERCICIOS

`model.Add(expr)`
Restricción aritmética o de igualdad

`model.AddAllDifferent(vars)`
Todas las variables deben ser distintas

`model.AddMaxEquality(t, lista)`
t = max(lista)

`model.AddAutomaton(vars, i, f, t)`
Restricción de autómata finito

`.OnlyEnforceIf(boolvar)`
Restricción condicional activada por BoolVar

`boolvar.Not()`
Negación de una BoolVar

RESTRICCIONES — NO USADAS EN EJERCICIOS +

`model.AddMinEquality(t, lista)`
t = min(lista)

`model.AddAbsEquality(t, var)`
t = |var|

`model.AddElement(idx, vars, t)`
t = vars[idx] (indexado por variable)

`model.AddBoolOr(lista)`
Al menos una BoolVar es verdadera

`model.AddBoolAnd(lista)`
Todas las BoolVar son verdaderas

`model.AddNoOverlap(intervalos)`
Intervalos sin solapamiento (scheduling)

RESOLVER Y EXTRAER RESULTADOS

`solver.Solve(model)`
Busca una solución óptima o factible

`solver.SearchForAllSolutions(m, cb)`
Enumera todas las soluciones

`solver.Value(var)`
Valor asignado a una variable

`cp.OPTIMAL / cp.FEASIBLE`
Constantes de status del solver

`solver.StatusName(status)`
Nombre legible del resultado

`solver.NumConflicts() / .WallTime()`
Métricas de rendimiento del solver

`solver.parameters.enumerate_all_solutions`
Activa enumeración completa de soluciones

DENTRO DEL SOLUTIONCALLBACK

`OnSolutionCallback(self)`
Método que se ejecuta en cada solución

`self.Value(var)`
Valor de variable dentro del callback

`self.StopSearch()`
Detiene la búsqueda desde el callback

numpy — import numpy as np

CONSTRUCCIÓN DE ARRAYS — USADAS EN EJERCICIOS

`np.array(lista_2d)`
Convierte lista de listas a ndarray

`np.zeros([n, n])`
Matriz inicial de ceros (grid vacío)

TRANSFORMACIONES DE MATRIZ — USADAS EN EJERCICIOS

`np_mtrx.transpose()`
Intercambia filas y columnas

`np.fliplr(m)`
Invierte columnas (izq ↔ der)

`np.flipud(m)`
Invierte filas (arriba ↔ abajo)

`m.flatten()`
Aplana a 1D (para AddAllDifferent)

`m.diagonal()`
Extrae la diagonal principal

OPERACIONES — USADAS EN EJERCICIOS

`np.sum(fila_o_col)`
Suma elementos (compatible con variables OR-T.)

ÚTILES NO USADAS EN EJERCICIOS +

`np.ones([n, n])`
Matriz de unos

`np.eye(n)`
Matriz identidad (diagonal de unos)

`np.rot90(m)`
Rota la matriz 90°

`m[i1:i2, j1:j2]`
Slicing de submatriz (celdas del sudoku)

`enumerate(np_mtrx)`
Itera filas con índice

Cada tarjeta es clickeable para explorar ese comando en detalle. Algunas notas sobre lo que encontré en tus archivos y lo que agregué:

Lo que sí aparece en los ejercicios son los comandos marcados sin etiqueta: toda la columna vertebral del patrón `CpModel → variables → restricciones → Solve → Value`. En numpy se usa principalmente para manipular matrices de variables OR-Tools, no para cálculo numérico.

Lo que agregué (marcado con +) son comandos que no aparecen en los archivos pero son muy comunes en la práctica:

En OR-Tools: `AddMinEquality`, `AddAbsEquality`, `AddElement` (indexar una lista con una variable), `AddBoolOr/And` (lógica proposicional directa), y `AddNoOverlap` (para problemas de scheduling como asignación de horarios). También `NewIntVarFromDomain` para dominios no contiguos como `{1,3,5,7}`.